

Materia: Autómatas, Teoría de Lenguajes y Compiladores	Código: 72.39
Créditos: 6	Carga horaria total: 102
Departamento: Sistemas Digitales y Datos	Año: 2023
Carrera de: Ingeniería en Informática	
Fecha última actualización: 21/12/2022 12:46:15	

➤ Carga horaria:

102	Horas totales
45	hs. teóricas
57	hs. prácticas
0	hs. de laboratorio

6	Horas semanales
6	hs. presencial
0	hs. a distancia

➤ Contenidos mínimos:

Lenguajes formales. Clasificación de Chomsky para los lenguajes formales. Autómata finito determinístico y no determinístico. Equivalencia entre ambos. Máquinas secuenciales. Lenguajes y expresiones regulares. Equivalencia entre expresiones regulares y autómatas finitos. Lemma de Pumping y aplicaciones. Gramáticas y lenguajes libres de contexto. Gramáticas y lenguajes sensibles al contexto. Máquina de Turing: su lenguaje y programación. Concepto básico de un compilador. Fases en la construcción de un compilador. Análisis léxico. Análisis sintáctico. Tabla de símbolos. Gramáticas con atributos. Generación de código intermedio. Uso de utilitarios tipo Lex & Yacc.

➤ Presentación de la materia:

Autómatas, Teoría de Lenguajes y Compiladores trata del estudio de máquinas abstractas que permiten modelar el funcionamiento de una computadora y de las gramáticas formales que permiten diseñar los lenguajes. El estudio de las máquinas abstractas permite determinar qué problemas se pueden resolver de forma eficiente mediante una computadora y cuáles son insolubles. El diseño de lenguajes y la construcción de compiladores permite implementar el desarrollo de software.

Se ubica en 3er año de la carrera y requiere como correlativa tener aprobada Estructura de Datos y Algoritmos (72.34), por lo que se asume que los alumnos al iniciar la cursada cuentan con conocimientos de programación en distintos paradigmas, de Matemática Discreta y de Álgebra. La materia ofrece los conceptos necesarios para abordar luego temas de Ciencias de la Computación. Se dicta en modalidad presencial, con 6 horas semanales.

La materia ofrece un abordaje dividido en tres aspectos: fundamentaciones teóricas, aplicaciones prácticas, y desarrollo de un proyecto de implementación. El objetivo final es que los alumnos puedan modelar lenguajes y las máquinas que los reconozcan según su tipo e implementar un analizador y compilador para alguno de esos

lenguajes.

➤ Objetivos de aprendizaje:

- Clasificar lenguajes según su tipo.
- Diseñar máquinas abstractas y gramáticas para un lenguaje.
- Analizar, evaluar e implementar un analizador y un compilador.
- Fundamentar con rigor matemático propiedades de los lenguajes y autómatas.
- Comprender y justificar formalmente las limitaciones que puede tener el software a la hora de resolver problemas.

➤ Contenidos:

Título	Descripción
I. Introducción	De qué se trata la materia. Conceptos fundamentales: Alfabeto y Cadena. Longitud de una cadena. La cadena vacía. Clausura + y clausura de Kleene. Concatenación de cadenas. Propiedades de la concatenación. Reverso de una cadena. Lenguajes. Operaciones con lenguajes. Definiciones recursivas. Demostraciones por inducción estructural.
II. Gramáticas	Definición de gramática. Derivaciones. Lenguaje generado por una gramática. Jerarquía de Chomsky. Gramáticas tipo 3, 2, 1 y 0. Lenguajes asociados a cada tipo de gramática: regulares, libres de contexto, dependientes del contexto y lenguajes tipo 0. Máquinas asociadas a cada tipo de gramática: autómatas finitos, autómatas de pila, máquinas de Turing.
III. Autómatas Finitos	Definición general. Función de transición y función de transición extendida. Configuración instantánea. Configuración inicial. Secuencia de configuraciones. Autómatas Finitos Deterministas (AFD). Lenguaje aceptado por un AFD. Equivalencia entre dos AFD. Relación de indistinguibilidad. Conjunto cociente. Minimización de un AFD. Autómatas Finitos No Deterministas (AFN). Lenguaje aceptado por un AFND. Equivalencia entre AFD y AFND. Autómatas Finitos No Deterministas Lambda (AFN-lambda). Lenguaje aceptado por un AFND-lambda. Clausuras lambda. Equivalencia entre AFD y AFND-lambda.
IV. Expresiones Regulares	Definición recursiva de una Expresión Regular. Lenguaje descrito por una Expresión Regular. Propiedades de las Expresiones Regulares. Equivalencia entre AFD y Expresiones Regulares. Las Expresiones Regulares en el desarrollo de un analizador-compilador. Análisis Léxico.
V. Lenguajes Regulares	Equivalencia entre los lenguajes generados por las Gramáticas Regulares, reconocidos por los Autómatas Finitos y descritos por las Expresiones Regulares. Lema de Bombeo para Lenguajes Regulares. Uso del lema para demostrar que un Lenguaje no es Regular.

	Propiedades de los Lenguajes Regulares. Problemas decidibles de los Lenguajes Regulares.
VI. Autómatas de Pila	Definición. Configuración instantánea. Configuración inicial. Secuencia de configuraciones. Lenguaje aceptado por un Autómata de Pila. Equivalencia entre autómatas que aceptan por pila vacía y autómatas que aceptan por estado final. Autómatas de Pila Deterministas (APD). Lenguajes aceptados por un APD.
VII. Gramáticas Libres de Contexto	Derivaciones más a izquierda y más a derecha. Lenguaje generado por una gramática. Lenguajes Libres de Contexto. Árboles de Derivación. Ambigüedad. Lenguajes inherentemente ambiguos.
VIII. Lenguajes Libres de Contexto	Equivalencia entre los lenguajes aceptados por los Autómatas de Pila y las Gramáticas Libres de Contexto. Uso de los Lenguajes Libres de Contexto en el desarrollo de lenguajes de programación. Formas Normales de Chomsky y de Greibach. Gramáticas Ambiguas y Autómatas de Pila Deterministas. Lema de Bombeo para Lenguajes Libres de Contexto. Uso del lema para demostrar que un Lenguaje no es Libre de Contexto. Propiedades de los Lenguajes Libres de Contexto. Problemas decidibles de los Lenguajes Libres de Contexto.
IX. Análisis Sintáctico	Métodos Descendentes de Análisis Sintáctico. Cálculo de los conjuntos de Primeros y Siguietes. Gramáticas LL(1). Análisis descendente recursivo. Análisis predictivo no recursivo. Métodos Ascendentes de Análisis Sintáctico. Reducciones. Análisis Sintáctico por Desplazamiento – Reducción. Analizadores sintácticos LR. Tablas de análisis. Prefijos viables. Conflictos. Analizadores sintácticos ascendentes más poderosos: LR(1) y LALR. Generadores de analizadores sintácticos: Yacc.
X. Gramáticas con Atributos	Atributos. Reglas semánticas. Atributos sintetizados y heredados. Definición Dirigida por la Sintaxis (DDS). Gramáticas L-atribuidas y S-atribuidas. Grafo de dependencias. Esquema de traducción. Esquema de un compilador. Chequeo de tipos con DDS. Generación de código intermedio con DDS.
XI. Máquinas de Turing	Definición. Descripciones instantáneas. Lenguaje de una Máquina de Turing. Máquina de Turing de varias cintas. Equivalencias entre Máquinas de Turing. Máquinas de Turing y computadoras. Problemas indecidibles para las máquinas de Turing.

➤ Estrategias de enseñanza:

- Exposición y demostración de conceptos y propiedades.
- Desarrollo de ejercicios que sirvan de ejemplo de los temas vistos.
- Planteo de situaciones problemáticas, exponiendo ejemplos sobre los cuales el alumno pueda empezar a desarrollar ideas propias.
- Corrección y orientación hacia el desarrollo de conceptos formales a partir de las soluciones planteadas por los alumnos.

Se procura un intercambio fluido con los docentes a través de la presencialidad y de los canales virtuales como campus y mail.

➤ Actividades:

Título	Descripción
TP01. Introducción.	Unidad I. Objetivo: - Adquirir la notación formal propia de la materia. - Efectuar demostraciones por inducción. Actividades: - Ejercicios de aplicación.
TP02. Gramáticas.	Unidad II. Objetivo: - Reconocer y clasificar gramáticas y los lenguajes que generan. Actividades: - Ejercicios de aplicación.
TP03. Autómatas Finitos.	Unidad III. Objetivo: - Escribir autómatas para reconocer lenguajes regulares. - Describir el funcionamiento de un autómata y el lenguaje que reconoce. - Obtener AFD mínimos. Actividades: - Ejercicios de aplicación.
TP04. Expresiones Regulares	Unidad IV. Objetivo: - Escribir expresiones regulares para reconocer lenguajes regulares. - Obtener expresiones regulares equivalentes. - Obtener equivalencias entre autómatas y expresiones regulares. Actividades: - Ejercicios de aplicación.

TP05. Lenguajes Regulares	Unidad V. Objetivo: - Usar autómatas, expresiones regulares o gramáticas para reconocer, describir o generar lenguajes regulares. - Demostrar que un lenguaje no es regular mediante el lema de bombeo. Actividades: - Ejercicios de aplicación.
TP06. Autómatas de Pila y Gramáticas Libres de Contexto	Unidad VI, VII, VIII Objetivo: - Escribir autómatas de pila o gramáticas libres de contexto para reconocer o generar lenguajes libres de contexto. - Describir el funcionamiento de un autómata y el lenguaje que reconoce. - Describir una gramática y el lenguaje que genera. - Reconocer gramáticas ambiguas. - Demostrar que un lenguaje no es libre de contexto mediante el lema de bombeo. Actividades: - Ejercicios de aplicación.
TP07. Análisis Sintáctico Descendente	Unidad IX Objetivo: - Construir un analizador sintáctico descendente para una gramática. - Reconocer gramáticas LL(1). Actividades: - Ejercicios de aplicación.
TP08. Análisis Sintáctico Ascendente	Unidad IX. Objetivo: - Construir un analizador sintáctico ascendente para una gramática. - Reconocer gramáticas LR(0), SLR, LALR Actividades: - Ejercicios de aplicación.
TP09. Gramáticas Con Atributos	Unidad X. Objetivo: - Construir una gramática con atributos identificando el tipo de traducción que requiere. Actividades: - Ejercicios de aplicación.
TP10. Máquinas de Turing	Unidad XI. Objetivo:

	- Escribir máquinas de Turing para reconocer lenguajes. - Describir el funcionamiento de una máquina de Turing. Actividades - Ejercicios de aplicación.
Proyecto Especial: Diseño e implementación de un lenguaje.	Unidades evaluadas: Unidades II, III, IV, V, VI, VII, VIII, IX, X Objetivo: - Diseñar y desarrollar un lenguaje y su compilador, construyendo sus dos componentes principales: el front-end y el back-end. Actividades: - Idear un lenguaje y sus prestaciones. - Redactar la gramática que defina su sintaxis. - Implementar una aplicación (compilador) que transformará un programa en un artefacto final sobre alguna plataforma. Entregas: Entrega 0: Conformación de grupos de hasta 4 integrantes y primera propuesta de lenguaje a implementar. Primera entrega: Front-end Segunda entrega: Back-end
Parcial 1	Unidades I, II, III, IV, V. Se espera que hayan logrado: - Adquirir la notación formal propia de la materia. - Demostrar adecuadamente por inducción. - Usar autómatas, expresiones regulares o gramáticas para reconocer, describir o generar lenguajes regulares. - Demostrar que un lenguaje no es regular mediante el lema de bombeo. Se evalúa por escrito mediante 4 o 5 ejercicios consistentes en demostraciones por inducción o lema de bombeo, obtención de autómatas mínimos o expresiones regulares o gramáticas para reconocer, describir o generar lenguajes regulares.
Parcial 2	Unidades VI, VII, VIII, IX, X, XI. Se espera que hayan logrado: - Escribir autómatas de pila o gramáticas libres de contexto para reconocer o generar lenguajes libres de contexto. - Describir el funcionamiento de un autómata y el lenguaje que reconoce. - Describir una gramática y el lenguaje que genera. - Demostrar que un lenguaje no es libre de contexto mediante el lema de bombeo. - Construir un analizador sintáctico descendente para una gramática. - Construir un analizador sintáctico ascendente para una gramática. - Escribir máquinas de Turing para reconocer lenguajes. Se evalúa por escrito mediante 4 o 5 ejercicios consistentes en demostración de lema de bombeo para lenguajes libres de contexto, hacer un analizador ascendente o descendente,

	obtención de autómatas de pila, gramáticas o máquinas de Turing para reconocer, o generar lenguajes
--	---

➤ Recursos didácticos:

Las clases son presenciales y como recursos se utilizan:

- pizarrón
- presentaciones digitalizadas
- videos y material teórico subidos a campus con anticipación.

➤ Modalidad de evaluación y aprobación:

Modalidad de evaluación:

- Parcial 1: Se realiza a mitad de cuatrimestre y es escrito (se encuentra detallado en la sección Actividades).
- Parcial 2: Se realiza a final de cuatrimestre y es escrito (se encuentra detallado en la sección Actividades).
- Proyecto de implementación: Se realiza en tres etapas durante el cuatrimestre, siendo la última hacia el final del cuatrimestre (se encuentra detallado en la sección Actividades).
- Recuperatorios: Si es necesario, se pueden recuperar los dos parciales y el Proyecto (en una única instancia cada uno).

Requisitos de aprobación:

- Parcial 1:(y recuperatorio) para aprobar debe sumar 6 puntos de 10.
- Parcial 2:(y recuperatorio) para aprobar debe sumar 6 puntos de 10.
- La aprobación del proyecto requiere de cumplir con las 3 entregas obligatorias.
- Se emplea una puntuación de 1 a 10 y para aprobar deb sumar 6 puntos de 10.
- Los criterios de evaluación son:
 - * Cumplimiento de todas las consignas de cada entrega.
 - * Implementación del compilador en cuanto a calidad técnica y creatividad.
 - * Calidad del informe y presentación del proyecto.
- La aprobación final de la cursada de la materia se da cuando se han aprobado los dos parciales y el proyecto (y sus recuperatorios si fuera necesario). La nota de cursada se calcula como el promedio de los parciales y el proyecto. En caso de aprobar en recuperatorio alguna instancia, se calcula la nota de esa instancia como 0.25 nota original+0.75 nota recuperatorio.
- La aprobación final de la materia se da con examen final, escrito, de contenido principalmente teórico. Para aprobar el examen, se exige una puntuación de 5 sobre 10.

➤ Bibliografía obligatoria:

N°	Descripción
----	-------------

1	J. E. Hopcroft, R. Motwani y J.D. Ullman. Introduction to Automata Theory, Languages and Computation (3ª ed.). USA. Addison—Wesley. 2006.
2	J. E. Hopcroft, R. Motwani y J.D. Ullman. Introducción a la teoría de autómatas, lenguajes y computación (3ª ed). España. Addison – Wesley. 2008.
3	J. E. Hopcroft y J.D. Ullman. Formal Languages and their Relation to Automata (1ª ed.) USA. Addison – Wesley. 1969.
4	A. V. Aho, M.S. Lam, R. Sethi y J.D. Ullman. Compilers: Principles, Techniques, and Tools (2ª ed). USA. Addison – Wesley. 2006.
5	A. V. Aho, The Theory of Parsing, Translation, and Compiling (Volume I: Parsing). USA. Prentice – Hall, 1972.

➤ **Bibliografía complementaria:**

N°	Descripción
1	K.D. Cooper y L. Torczon. Engineering a Compiler (2ª ed.) USA. Elsevier. 2011.
2	J.R. Levine. Flex y Bison (1ª ed). USA. O'Reilly Media Inc. 2009.
3	M. Sipser. Introduction to the Theory of Computation (3ª ed). USA. CENGAGE Learning. 2012.